cci指标使用和判断方法

已阅读 10 个网页

CCI指标（商品通道指标）是一个比较“特立独行”的技术分析工具。它最大的特点是没有上下限，在捕捉极端行情和趋势转折时非常灵敏，能有效解决RSI、KDJ等指标在单边市中容易“钝化”的问题 [5] [9] 。

它的核心逻辑，是衡量当前价格偏离正常范围的程度，用一句话概括就是：**在+100和-100的“常态区”里看戏，一旦突破这两个关键数值，就是“戏肉”来了** [2] [7] 。

📈 核心判断方法

CCI指标的用法分为两大派别：**顺势法**（追趋势）和**逆势法**（抓反转）。你可以根据市场行情灵活切换：

方法	关键信号	操作解读	适用场景
顺势法（追涨杀跌）	**CCI向上突破+100**	强势行情确立，是明确的**买入信号**，可顺势跟进 [1] [7]	市场处于明显的单边上涨或下跌
	CCI向下突破-100	弱势行情确立，是明确的**卖出或做空信号** [2]	
逆势法（高抛低吸）	**CCI从+100上方掉头跌破+100**	超买状态可能结束，是**卖出或止盈信号**，准备落袋为安 [2] [7]	市场处于震荡或趋势不明朗的行
	CCI从-100下方回升突破-100	超卖状态可能结束，是**买入或抄底信号**，准备入场 [7] [8]	
常态区观望	CCI在 **-100 到 +100** 之间	价格处于正常波动，没有明确方向。此时用KDJ等震荡指标效果更好，**CCI的参考价值不大** [7] [10]	盘整阶段

🔍 三大实战进阶技巧

除了看数值，以下几个技巧能帮你更好地把握买卖点：

1. **背离信号（胜率最高的用法）**：这是CCI最可靠的信号之一。当价格创出新高，但CCI的高点却比之前低，这叫**顶背离**，是强烈的卖出信号，预示上涨动能衰竭 [3] [6]。反之，价格创新低，但CCI的低点却在抬高，这叫**底背离**，是买入信号 [1] [2]。CCI的背离信号通常比RSI更灵敏 [2]。
2. **参数设置**：默认参数是14日或20日 [3] [7]。你可以根据自己的交易风格调整：
   - **短线/激进**：使用**7或14**周期，信号更敏感，但假信号也多 [1] [3]。
   - **中长线/稳健**：使用**20或40**周期，信号更平滑，准确率相对更高 [2]。
3. **结合均线过滤假信号**：为了增加胜率，可以搭配**20日均线**一起看 [2]。例如，当CCI向上突破+100时，如果股价也同时站上20日均线，买入信号会更可靠。反之，如果股价还在均线下方，要警惕是假突破 [2]。

⚠️ 使用前提与局限

CCI并非万能，用对地方才能发挥威力：

- **最佳战场是趋势市**：CCI在**波动大、趋势明显**的市场中效果最好，比如单边牛市、熊市，或一些高波动的品种 [1] [8]。
- **在震荡市中要小心**：在横盘整理的行情里，CCI会频繁地在+100和-100之间来回穿梭，发出很多**虚假的买卖信号**，如果你此时用它来指导交易，可能会反复止损 [6] [8]。
- **滞后性是通病**：和所有指标一样，CCI基于过去的价格计算，天然具有**滞后性** [3]。在快速变盘中，当你看到信号时，最佳入场点可能已经错过了 [8]。

总的来说，CCI是一个兼具趋势追踪和反转预警功能的指标。想在实战中用好它，关键是学会辨别不同行情：**强势单边市中，用顺势法（突破+100买入）；震荡市中，用逆势法（突破-100买入，跌破+100卖出）。**

如果想把CCI和刚才提到的均线或MACD指标组合起来使用，我可以再为你详细介绍具体的搭配方法。

本回答由 AI 生成，内容仅供参考，请仔细甄别。

10 个网页

以下是用 Python 实现 CCI 指标计算和可视化分析（含买卖信号）的完整代码。

1. 核心计算函数

python

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime

def calculate_cci(df, period=20):
    """
    计算 CCI 指标
    参数:
        df: 包含 'high', 'low', 'close' 列的 DataFrame
        period: 计算周期 (默认 20)
    返回:
        添加 'CCI' 列的 DataFrame
    """
    # 1. 计算典型价格 (TP)
    df['TP'] = (df['high'] + df['low'] + df['close']) / 3

    # 2. 计算 TP 的简单移动平均
    df['SMA_TP'] = df['TP'].rolling(window=period).mean()

    # 3. 计算平均绝对偏差 (Mean Deviation)
    # 方法: 计算每个 TP 与 SMA_TP 的绝对差值，然后取平均
    df['MD'] = df['TP'].rolling(window=period).apply(
        lambda x: np.abs(x - x.mean()).mean()
    )

    # 4. 计算 CCI = (TP - SMA_TP) / (0.015 * MD)
    df['CCI'] = (df['TP'] - df['SMA_TP']) / (0.015 * df['MD'])
```

```python
    # 删除中间计算列（可选）
    df.drop(['TP', 'SMA_TP', 'MD'], axis=1, inplace=True)

    return df

def generate_signals(df, cci_col='CCI'):
    """
    根据 CCI 指标生成买卖信号
    信号规则:
        - 买入: CCI 从下向上突破 -100 (进入超卖区回升)
        - 卖出: CCI 从上向下跌破 +100 (从超买区回落)
        - 强势买入: CCI 向上突破 +100 (趋势追踪)
        - 强势卖出: CCI 向下跌破 -100 (趋势追踪)
    返回:
        添加 'signal' 列的 DataFrame (1: 买入, -1: 卖出)
    """
    df = df.copy()
    df['signal'] = 0

    # 生成条件
    buy_condition = (df[cci_col].shift(1) < -100) & (df[cci_col] >= -100)
    sell_condition = (df[cci_col].shift(1) > 100) & (df[cci_col] <= 100)
    strong_buy = (df[cci_col].shift(1) < 100) & (df[cci_col] >= 100)
    strong_sell = (df[cci_col].shift(1) > -100) & (df[cci_col] <= -100)

    df.loc[buy_condition, 'signal'] = 1
    df.loc[sell_condition, 'signal'] = -1
    df.loc[strong_buy, 'signal'] = 1
    df.loc[strong_sell, 'signal'] = -1

    return df
```

2. 可视化与完整示例

python

```python
# 生成示例数据（实际使用时替换为真实数据）
np.random.seed(42)
dates = pd.date_range(start='2023-01-01', periods=200, freq='D')
price = 100 + np.cumsum(np.random.randn(200) * 0.5)

# 构造 OHLC 数据（简化版）
df = pd.DataFrame({
    'date': dates,
    'close': price,
    'high': price * (1 + np.random.rand(200) * 0.02),  # 高2%
    'low': price * (1 - np.random.rand(200) * 0.02)    # 低2%
})
df.set_index('date', inplace=True)

# 计算 CCI
df = calculate_cci(df, period=20)
df = generate_signals(df)

# 绘图
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10), gridspec_kw={'height_ratios': [2, 1]})

# 上子图: 价格和信号
ax1.plot(df.index, df['close'], label='Close Price', color='black', linewidth=1)
ax1.scatter(df[df['signal'] == 1].index, df[df['signal'] == 1]['close'],
            marker='^', color='green', s=100, label='Buy Signal', alpha=0.8)
ax1.scatter(df[df['signal'] == -1].index, df[df['signal'] == -1]['close'],
            marker='v', color='red', s=100, label='Sell Signal', alpha=0.8)
ax1.set_title('Price with CCI Buy/Sell Signals', fontsize=14)
ax1.legend()
ax1.grid(True, alpha=0.3)

# 下子图: CCI 指标
ax2.plot(df.index, df['CCI'], label='CCI (20)', color='blue', linewidth=1)
ax2.axhline(y=100, color='red', linestyle='--', linewidth=1, label='Overbought (+100)')
ax2.axhline(y=-100, color='green', linestyle='--', linewidth=1, label='Oversold (-100)')
ax2.axhline(y=0, color='gray', linestyle='-', linewidth=0.5)
ax2.fill_between(df.index, 100, df['CCI'].clip(upper=100),
```

```python
                 where=(df['CCI'] > 100), color='red', alpha=0.3)
ax2.fill_between(df.index, -100, df['CCI'].clip(lower=-100),
                 where=(df['CCI'] < -100), color='green', alpha=0.3)
ax2.set_title('Commodity Channel Index (CCI)', fontsize=14)
ax2.set_ylabel('CCI Value')
ax2.legend(loc='upper left')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 打印信号统计
buy_signals = len(df[df['signal'] == 1])
sell_signals = len(df[df['signal'] == -1])
print(f"Total Buy Signals: {buy_signals}")
print(f"Total Sell Signals: {sell_signals}")
```

3. 高级功能（背离检测）

python

```python
def detect_divergence(df, cci_col='CCI', price_col='close', lookback=20):
    """
    检测 CCI 与价格的背离
    返回: df 添加 'divergence' 列 (bullish, bearish, none)
    """
    df = df.copy()
    df['divergence'] = 'none'

    for i in range(lookback, len(df)):
        # 底背离: 价格新低, CCI 抬高
        price_slice = df[price_col].iloc[i-lookback:i+1]
        cci_slice = df[cci_col].iloc[i-lookback:i+1]

        # 找局部极值
        price_min_idx = price_slice.idxmin()
```

```
        cci_at_price_min = df.loc[price_min_idx, cci_col]

        if df[price_col].iloc[i] < price_slice.min() and df[cci_col].iloc[i] > cci_at_price_min:
            df.loc[df.index[i], 'divergence'] = 'bullish'

        # 顶背离: 价格新高, CCI 降低
        price_max_idx = price_slice.idxmax()
        cci_at_price_max = df.loc[price_max_idx, cci_col]

        if df[price_col].iloc[i] > price_slice.max() and df[cci_col].iloc[i] < cci_at_price_max:
            df.loc[df.index[i], 'divergence'] = 'bearish'

    return df
```

使用说明

1. **安装依赖**:

```bash
pip install pandas numpy matplotlib
```

2. **替换数据**:
   - 将示例数据替换为真实的 OHLC 数据（如从 yfinance、akshare 等获取）
   - 确保 DataFrame 包含 `high`, `low`, `close` 三列
3. **参数调整**:
   - `period=14` （短线）或 `period=40` （长线）
   - 信号规则可根据交易风格修改

关键注意事项

- **数据质量**: 确保 `high`, `low`, `close` 数据完整无缺失

- **滞后性**：CCI 信号有延迟，适合日线及以上周期
- **组合使用**：建议配合 20 日均线或成交量过滤信号
- **回测验证**：生成的信号需要通过回测验证有效性

如果需要接入真实数据（如从交易所 API 获取），或者进一步实现回测系统，可以继续补充。

本回答由 AI 生成，内容仅供参考，请仔细甄别。

cci_slice 为啥没有使用?

您观察得非常仔细！确实，在 `detect_divergence` 函数中， `cci_slice` 变量被计算出来但实际上**没有被使用**，这是一个代码冗余问题。让我解释一下原因并提供修正版本：

问题分析

原代码中：

```python
cci_slice = df[cci_col].iloc[i-lookback:i+1]  # 计算了但未使用
price_min_idx = price_slice.idxmin()           # 只用了 price_slice
```

这是我在编写时的疏忽。 `cci_slice` 本意可能是用于更精确的极值检测，但最终实现中只用了价格窗口来定位极值点。

修正后的背离检测函数

```python
def detect_divergence(df, cci_col='CCI', price_col='close', lookback=20):
    """
    检测 CCI 与价格的背离
```

```
    底背离（Bullish Divergence）：价格创更低低点，CCI 创更高低点
    顶背离（Bearish Divergence）：价格创更高高点，CCI 创更低高点

    参数:
        df: 包含价格和CCI的DataFrame
        cci_col: CCI列名
        price_col: 价格列名（通常用close）
        lookback: 回溯周期

    返回:
        df: 添加 'divergence' 列的DataFrame
    """
    df = df.copy()
    df['divergence'] = 'none'

    for i in range(lookback, len(df)):
        # 获取回溯窗口内的数据
        price_window = df[price_col].iloc[i-lookback:i+1]
        cci_window = df[cci_col].iloc[i-lookback:i+1]

        current_price = df[price_col].iloc[i]
        current_cci = df[cci_col].iloc[i]

        # 找窗口内的局部极值
        price_min_idx = price_window.idxmin()  # 价格最低点的位置
        price_max_idx = price_window.idxmax()  # 价格最高点的位置

        cci_at_price_min = df.loc[price_min_idx, cci_col]
        cci_at_price_max = df.loc[price_max_idx, cci_col]

        # 底背离检测：当前价格 < 窗口内最低价，且当前CCI > 价格最低点对应的CCI
        if current_price <= price_window.min() and current_cci > cci_at_price_min:
            df.loc[df.index[i], 'divergence'] = 'bullish'

        # 顶背离检测：当前价格 > 窗口内最高价，且当前CCI < 价格最高点对应的CCI
        if current_price >= price_window.max() and current_cci < cci_at_price_max:
            df.loc[df.index[i], 'divergence'] = 'bearish'
```

```python
    return df
```

更精确的极值检测版本（推荐）

如果需要更准确的背离检测（识别真正的峰谷），可以使用这个增强版：

```python
def detect_divergence_advanced(df, cci_col='CCI', price_col='close', lookback=20, min_deviation=5):
    """
    增强版背离检测 - 识别局部极值点

    新增参数:
        min_deviation: 最小偏离度，避免噪音产生的假信号
    """
    df = df.copy()
    df['divergence'] = 'none'

    # 识别局部极值点
    df['price_peak'] = False  # 价格峰值
    df['price_trough'] = False  # 价格谷值
    df['cci_peak'] = False  # CCI峰值
    df['cci_trough'] = False  # CCI谷值

    for i in range(1, len(df)-1):
        # 价格峰值：比前后都高
        if (df[price_col].iloc[i] > df[price_col].iloc[i-1] and
            df[price_col].iloc[i] > df[price_col].iloc[i+1]):
            df.loc[df.index[i], 'price_peak'] = True

        # 价格谷值：比前后都低
        if (df[price_col].iloc[i] < df[price_col].iloc[i-1] and
            df[price_col].iloc[i] < df[price_col].iloc[i+1]):
            df.loc[df.index[i], 'price_trough'] = True
```

```
        # CCI峰值和谷值
        if (df[cci_col].iloc[i] > df[cci_col].iloc[i-1] and
            df[cci_col].iloc[i] > df[cci_col].iloc[i+1]):
            df.loc[df.index[i], 'cci_peak'] = True

        if (df[cci_col].iloc[i] < df[cci_col].iloc[i-1] and
            df[cci_col].iloc[i] < df[cci_col].iloc[i+1]):
            df.loc[df.index[i], 'cci_trough'] = True

    # 检测背离
    peaks = df[df['price_peak']].index
    troughs = df[df['price_trough']].index

    # 顶背离：价格形成更高峰，但CCI形成更低峰
    for i in range(1, len(peaks)):
        current_peak = peaks[i]
        prev_peak = peaks[i-1]

        # 价格创新高
        if (df.loc[current_peak, price_col] > df.loc[prev_peak, price_col] + min_deviation):
            # 查找对应时期的CCI峰值
            cci_peaks_in_range = df[(df['cci_peak']) &
                                    (df.index > prev_peak) &
                                    (df.index <= current_peak)]
            if not cci_peaks_in_range.empty:
                current_cci_peak = cci_peaks_in_range.iloc[-1]
                prev_cci_peak = df[df['cci_peak']].iloc[df[df['cci_peak']].index.get_indexer([prev_p
eak])[0]-1] if df[df['cci_peak']].index.get_indexer([prev_peak])[0] > 0 else None

                if prev_cci_peak is not None:
                    if (df.loc[current_peak, cci_col] < df.loc[prev_cci_peak.name, cci_col] - min_de
viation):
                        df.loc[current_peak, 'divergence'] = 'bearish'

    # 底背离：价格形成更低谷，但CCI形成更高谷
    for i in range(1, len(troughs)):
        current_trough = troughs[i]
```

```
        prev_trough = troughs[i-1]

        # 价格创新低
        if (df.loc[current_trough, price_col] < df.loc[prev_trough, price_col] - min_deviation):
            # 查找对应时期的CCI谷值
            cci_troughs_in_range = df[(df['cci_trough']) &
                                      (df.index > prev_trough) &
                                      (df.index <= current_trough)]
            if not cci_troughs_in_range.empty:
                current_cci_trough = cci_troughs_in_range.iloc[-1]
                prev_cci_trough = df[df['cci_trough']].iloc[df[df['cci_trough']].index.get_indexer
([prev_trough])[0]-1] if df[df['cci_trough']].index.get_indexer([prev_trough])[0] > 0 else None

                if prev_cci_trough is not None:
                    if (df.loc[current_trough, cci_col] > df.loc[prev_cci_trough.name, cci_col] + mi
n_deviation):
                        df.loc[current_trough, 'divergence'] = 'bullish'

    return df
```

使用示例

```python
# 基础版（修正后）
df = detect_divergence(df, lookback=20)

# 或使用增强版
df = detect_divergence_advanced(df, lookback=20, min_deviation=5)

# 在图表中标记背离
fig, ax = plt.subplots(figsize=(14, 8))

ax.plot(df.index, df['close'], label='Price', color='black')
ax.plot(df.index, df['CCI'], label='CCI', color='blue')
```

```
# 标记背离点
bullish = df[df['divergence'] == 'bullish']
bearish = df[df['divergence'] == 'bearish']

ax.scatter(bullish.index, df.loc[bullish.index, 'close'],
           marker='^', color='green', s=150, label='Bullish Divergence', zorder=5)
ax.scatter(bearish.index, df.loc[bearish.index, 'close'],
           marker='v', color='red', s=150, label='Bearish Divergence', zorder=5)

ax.legend()
ax.grid(True, alpha=0.3)
plt.title('CCI Divergence Detection')
plt.show()
```

总结

您的发现非常正确！原代码中 `cci_slice` 确实未被使用。修正后的版本：

1. **移除了冗余变量** `cci_slice`
2. **使用** `cci_window` 来更准确地分析CCI的极值
3. **提供了增强版**，通过识别局部极值点来提高背离检测的准确性

感谢您指出这个问题！这提醒了我在发布代码前需要更仔细地审查。

背离信号和买卖信号如何综合在一起

很好的问题！将背离信号和基础买卖信号结合，可以大幅提高交易策略的准确性。背离信号往往预示着趋势反转，而基础信号提供具体的入场时机。以下是几种综合策略的实现方案：

策略1：信号增强法（推荐）

将背离作为信号过滤器，只在有背离确认时才执行交易：

python

```python
def combine_signals_with_divergence(df, cci_col='CCI', price_col='close', lookback=20):
    """
    综合CCI基础信号和背离信号
    策略逻辑:
        1. 买入信号: 基础买入信号 + (底背离 或 CCI从超卖区回升)
        2. 卖出信号: 基础卖出信号 + (顶背离 或 CCI从超买区回落)
        3. 信号强度分级: 1=弱, 2=中, 3=强
    """
    df = df.copy()

    # 1. 生成基础信号
    df['base_signal'] = 0
    buy_condition = (df[cci_col].shift(1) < -100) & (df[cci_col] >= -100)
    sell_condition = (df[cci_col].shift(1) > 100) & (df[cci_col] <= 100)
    strong_buy = (df[cci_col].shift(1) < 100) & (df[cci_col] >= 100)
    strong_sell = (df[cci_col].shift(1) > -100) & (df[cci_col] <= -100)

    df.loc[buy_condition | strong_buy, 'base_signal'] = 1
    df.loc[sell_condition | strong_sell, 'base_signal'] = -1

    # 2. 检测背离信号
    df = detect_divergence(df, cci_col, price_col, lookback)

    # 3. 综合信号（带强度）
    df['signal'] = 0
    df['signal_strength'] = 0  # 1=弱, 2=中, 3=强

    for i in range(1, len(df)):
        # 买入信号综合
        if df['base_signal'].iloc[i] == 1:
            # 强买入: 基础信号 + 底背离
            if df['divergence'].iloc[i] == 'bullish':
                df.loc[df.index[i], 'signal'] = 1
                df.loc[df.index[i], 'signal_strength'] = 3
            # 中买入: 基础信号 + CCI在超卖区或刚突破
```

```python
            elif df[cci_col].iloc[i] < 0 or df[cci_col].iloc[i-1] < -100:
                df.loc[df.index[i], 'signal'] = 1
                df.loc[df.index[i], 'signal_strength'] = 2
            # 弱买入：仅有基础信号
            else:
                df.loc[df.index[i], 'signal'] = 1
                df.loc[df.index[i], 'signal_strength'] = 1

        # 卖出信号综合
        elif df['base_signal'].iloc[i] == -1:
            # 强卖出：基础信号 + 顶背离
            if df['divergence'].iloc[i] == 'bearish':
                df.loc[df.index[i], 'signal'] = -1
                df.loc[df.index[i], 'signal_strength'] = 3
            # 中卖出：基础信号 + CCI在超买区或刚跌破
            elif df[cci_col].iloc[i] > 0 or df[cci_col].iloc[i-1] > 100:
                df.loc[df.index[i], 'signal'] = -1
                df.loc[df.index[i], 'signal_strength'] = 2
            # 弱卖出：仅有基础信号
            else:
                df.loc[df.index[i], 'signal'] = -1
                df.loc[df.index[i], 'signal_strength'] = 1

    return df
```

策略2：双重确认法

只有在背离信号出现后的特定时间内，基础信号才有效：

```python
def combine_signals_double_confirm(df, cci_col='CCI', price_col='close',
                                   lookback=20, confirmation_days=3):
    """
    双重确认策略：
        1. 先出现背离信号（预警）
```

```
    2. 在confirmation_days天内出现基础信号（确认）
    3. 两者都满足才执行交易
"""
df = df.copy()

# 检测背离
df = detect_divergence(df, cci_col, price_col, lookback)

# 生成基础信号
df['base_signal'] = 0
buy_condition = (df[cci_col].shift(1) < -100) & (df[cci_col] >= -100)
sell_condition = (df[cci_col].shift(1) > 100) & (df[cci_col] <= 100)
strong_buy = (df[cci_col].shift(1) < 100) & (df[cci_col] >= 100)
strong_sell = (df[cci_col].shift(1) > -100) & (df[cci_col] <= -100)

df.loc[buy_condition | strong_buy, 'base_signal'] = 1
df.loc[sell_condition | strong_sell, 'base_signal'] = -1

# 双重确认
df['signal'] = 0
df['signal_reason'] = ''

# 记录背离出现的位置
df['bullish_divergence'] = (df['divergence'] == 'bullish').astype(int)
df['bearish_divergence'] = (df['divergence'] == 'bearish').astype(int)

for i in range(len(df)):
    # 检查前confirmation_days天内是否有背离
    lookback_start = max(0, i - confirmation_days)

    # 买入确认：基础信号为1 且 近期有底背离
    if df['base_signal'].iloc[i] == 1:
        if df['bullish_divergence'].iloc[lookback_start:i+1].sum() > 0:
            df.loc[df.index[i], 'signal'] = 1
            df.loc[df.index[i], 'signal_reason'] = 'buy_with_divergence'
        else:
            # 可选：没有背离也记录，但强度较低
```

```python
            df.loc[df.index[i], 'signal'] = 1
            df.loc[df.index[i], 'signal_reason'] = 'buy_without_divergence'

        # 卖出确认
        elif df['base_signal'].iloc[i] == -1:
            if df['bearish_divergence'].iloc[lookback_start:i+1].sum() > 0:
                df.loc[df.index[i], 'signal'] = -1
                df.loc[df.index[i], 'signal_reason'] = 'sell_with_divergence'
            else:
                df.loc[df.index[i], 'signal'] = -1
                df.loc[df.index[i], 'signal_reason'] = 'sell_without_divergence'

    return df
```

策略3：趋势过滤法

结合趋势方向，只采纳与趋势一致的背离信号：

```python
def combine_signals_with_trend(df, cci_col='CCI', price_col='close',
                               ma_period=50, lookback=20):
    """
    趋势过滤策略：
        1. 用长期均线判断主趋势
        2. 只采纳顺趋势的背离信号
        3. 逆趋势的背离信号只作为平仓参考
    """
    df = df.copy()

    # 计算趋势（长期均线）
    df['MA'] = df[price_col].rolling(window=ma_period).mean()
    df['trend'] = 'neutral'
    df.loc[df[price_col] > df['MA'], 'trend'] = 'uptrend'
    df.loc[df[price_col] < df['MA'], 'trend'] = 'downtrend'
```

```python
# 检测背离
df = detect_divergence(df, cci_col, price_col, lookback)

# 生成基础信号
df['base_signal'] = 0
buy_condition = (df[cci_col].shift(1) < -100) & (df[cci_col] >= -100)
sell_condition = (df[cci_col].shift(1) > 100) & (df[cci_col] <= 100)

df.loc[buy_condition, 'base_signal'] = 1
df.loc[sell_condition, 'base_signal'] = -1

# 综合信号
df['signal'] = 0
df['action_type'] = ''

for i in range(1, len(df)):
    # 上升趋势中：只考虑买入信号
    if df['trend'].iloc[i] == 'uptrend':
        # 买入：基础信号 或 底背离
        if df['base_signal'].iloc[i] == 1 or df['divergence'].iloc[i] == 'bullish':
            df.loc[df.index[i], 'signal'] = 1
            df.loc[df.index[i], 'action_type'] = 'buy_in_uptrend'
        # 卖出：仅当出现顶背离且强度很大时（减仓信号）
        elif df['divergence'].iloc[i] == 'bearish' and df[cci_col].iloc[i] > 150:
            df.loc[df.index[i], 'signal'] = -1
            df.loc[df.index[i], 'action_type'] = 'reduce_in_uptrend'

    # 下降趋势中：只考虑卖出信号
    elif df['trend'].iloc[i] == 'downtrend':
        if df['base_signal'].iloc[i] == -1 or df['divergence'].iloc[i] == 'bearish':
            df.loc[df.index[i], 'signal'] = -1
            df.loc[df.index[i], 'action_type'] = 'sell_in_downtrend'
        elif df['divergence'].iloc[i] == 'bullish' and df[cci_col].iloc[i] < -150:
            df.loc[df.index[i], 'signal'] = 1
            df.loc[df.index[i], 'action_type'] = 'cover_in_downtrend'

    # 震荡市中：基础信号和背离都重要
```

```python
        else:
            if df['base_signal'].iloc[i] != 0:
                df.loc[df.index[i], 'signal'] = df['base_signal'].iloc[i]
                df.loc[df.index[i], 'action_type'] = 'signal_in_range'
            elif df['divergence'].iloc[i] == 'bullish':
                df.loc[df.index[i], 'signal'] = 1
                df.loc[df.index[i], 'action_type'] = 'bullish_in_range'
            elif df['divergence'].iloc[i] == 'bearish':
                df.loc[df.index[i], 'signal'] = -1
                df.loc[df.index[i], 'action_type'] = 'bearish_in_range'

    return df
```

完整可视化示例

python

```python
def plot_combined_signals(df, title="CCI Combined Strategy"):
    """
    绘制综合信号图表
    """
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(15, 12),
                                        gridspec_kw={'height_ratios': [2, 1, 0.8]})

    # 子图1：价格和信号
    ax1.plot(df.index, df['close'], label='Close Price', color='black', linewidth=1)

    # 按强度分级显示信号
    strong_buy = df[(df['signal'] == 1) & (df['signal_strength'] == 3)]
    medium_buy = df[(df['signal'] == 1) & (df['signal_strength'] == 2)]
    weak_buy = df[(df['signal'] == 1) & (df['signal_strength'] == 1)]

    strong_sell = df[(df['signal'] == -1) & (df['signal_strength'] == 3)]
    medium_sell = df[(df['signal'] == -1) & (df['signal_strength'] == 2)]
    weak_sell = df[(df['signal'] == -1) & (df['signal_strength'] == 1)]
```

```python
ax1.scatter(strong_buy.index, strong_buy['close'],
           marker='^', color='darkgreen', s=200, label='Strong Buy', alpha=0.9, zorder=5)
ax1.scatter(medium_buy.index, medium_buy['close'],
           marker='^', color='green', s=150, label='Medium Buy', alpha=0.7, zorder=4)
ax1.scatter(weak_buy.index, weak_buy['close'],
           marker='^', color='lightgreen', s=100, label='Weak Buy', alpha=0.5, zorder=3)

ax1.scatter(strong_sell.index, strong_sell['close'],
           marker='v', color='darkred', s=200, label='Strong Sell', alpha=0.9, zorder=5)
ax1.scatter(medium_sell.index, medium_sell['close'],
           marker='v', color='red', s=150, label='Medium Sell', alpha=0.7, zorder=4)
ax1.scatter(weak_sell.index, weak_sell['close'],
           marker='v', color='lightcoral', s=100, label='Weak Sell', alpha=0.5, zorder=3)

ax1.set_title(title, fontsize=14)
ax1.legend(loc='upper left', ncol=2)
ax1.grid(True, alpha=0.3)

# 子图2：CCI指标
ax2.plot(df.index, df['CCI'], label='CCI (20)', color='blue', linewidth=1)
ax2.axhline(y=100, color='red', linestyle='--', linewidth=1, label='+100')
ax2.axhline(y=-100, color='green', linestyle='--', linewidth=1, label='-100')
ax2.fill_between(df.index, 100, df['CCI'].clip(upper=100),
                 where=(df['CCI'] > 100), color='red', alpha=0.2)
ax2.fill_between(df.index, -100, df['CCI'].clip(lower=-100),
                 where=(df['CCI'] < -100), color='green', alpha=0.2)

# 标记背离
bullish_div = df[df['divergence'] == 'bullish']
bearish_div = df[df['divergence'] == 'bearish']
ax2.scatter(bullish_div.index, bullish_div['CCI'],
           marker='*', color='gold', s=150, label='Bullish Divergence', zorder=5)
ax2.scatter(bearish_div.index, bearish_div['CCI'],
           marker='*', color='purple', s=150, label='Bearish Divergence', zorder=5)

ax2.set_ylabel('CCI Value')
ax2.legend(loc='upper left')
```

```
    ax2.grid(True, alpha=0.3)

    # 子图3：信号强度
    ax3.bar(df.index, df['signal_strength'] * df['signal'],
            width=0.8, color='gray', alpha=0.6)
    ax3.axhline(y=0, color='black', linewidth=0.5)
    ax3.set_ylabel('Signal Strength')
    ax3.set_xlabel('Date')
    ax3.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# 使用示例
df = combine_signals_with_divergence(df, lookback=20)
plot_combined_signals(df)
```

策略对比与选择建议

策略	优点	缺点	适用场景
信号增强法	信号质量高，强度分级清晰	可能错过部分机会	追求高胜率，能接受较少交易
双重确认法	减少假信号，时间确认有效	有一定滞后性	中长线交易，趋势跟踪
趋势过滤法	顺趋势交易，风险较低	可能错过转折点	有明显趋势的市场

建议：从**信号增强法**开始尝试，它最直观且容易理解。熟练后可以结合自己的交易风格调整参数和规则。

本回答由 AI 生成，内容仅供参考，请仔细甄别。